ComPASS: A Compatible PIM Protocol Architecture and Scheduling Solution for Processor-PIM Collaboration

Seunghyuk Yu
Inha University
Incheon, Republic of Korea
dodo7se@gmail.com

Hyeonu Kim
Inha University
Incheon, Republic of Korea
lionkim2006@gmail.com

Kyoungho Jeun
Inha University
Incheon, Republic of Korea
qldkssp99@gmail.com

Sunyoung Hwang
Inha University
Incheon, Republic of Korea
as7425135@gmail.com

Seongmin Cho
Inha University
Incheon, Republic of Korea
whtjdals0704@gmail.com

Eojin Lee
Inha University
Incheon, Republic of Korea
ejlee@inha.ac.kr

Abstract

With growing demands from memory-bound applications, Processing-In-Memory (PIM) architectures have emerged as a promising way to reduce data movement. However, existing PIM designs face challenges in compatibility and efficiency due to limited command/address space and the overhead of switching between PIM and normal memory modes. To address these limitations, we propose ComPASS, a compatible PIM protocol and scheduling solution that enables efficient processor-PIM integration. ComPASS introduces PIM-ACT, a new memory command that triggers multi-bank activation and PIM operations at once, allowing for compatibility across diverse PIM architectures. It also adds a PIM request generator within the memory controller, which can serve all PIM devices compliant with PIM-ACT, avoiding device-specific integration. In addition, ComPASS supports architecture-aware optimizations, enabling each PIM device to use a customized PIM-ACT configuration tailored to its architecture without compromising compatibility. For efficient scheduling of both PIM and conventional memory requests, ComPASS proposes static and adaptive scheduling that maintain PIM throughput while allocating bandwidth to conventional workloads for overall system efficiency. Our evaluation shows that ComPASS consistently meets PIM performance targets by achieving up to 10.75× GEMV speedup on LPDDR-PIM over non-PIM environments, even under co-execution with memory-intensive CPU workloads, thereby demonstrating both efficiency and compatibility.

Keywords

Processing-in-memory, Memory protocol, Memory scheduling, Memory command

ACM Reference Format:
Seunghyuk Yu, Hyeonu Kim, Kyoungho Jeun, Sunyoung Hwang, Seongmin Cho, and Eojin Lee. 2025. ComPASS: A Compatible PIM Protocol Architecture and Scheduling Solution for Processor-PIM Collaboration. In *58th IEEE/ACM International Symposium on Microarchitecture (MICRO '25), October 18–22, 2025, Seoul, Republic of Korea.* ACM, New York, NY, USA, 14 pages. https://doi.org/10.1145/3725843.3756017

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
MICRO '25, Seoul, Republic of Korea
© 2025 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 979-8-4007-1573-0/25/10
https://doi.org/10.1145/3725843.3756017

1 Introduction

Processing-In-Memory (PIM [1, 2, 5, 8, 12, 16, 18, 25, 29, 48, 52, 55, 58, 61, 64]) presents a promising solution to address the memory bandwidth limitation in memory-intensive operations, such as general matrix-vector multiplication (GEMV), which is widely used in large language models (LLMs [4, 6, 11, 57, 62]) inference. Unlike conventional architectures that require data transfers from memory to processor, PIM architectures implement computational capabilities directly within memory arrays. They mitigate data movement overhead between memory and processors, yielding significant performance improvements. For example, to perform GEMV operation, traditional architectures must repeatedly fetch non-reusable matrix data from memory, whereas PIM architectures leverage memory internal bandwidth by computing in multiple processing units (PUs) within each bank using the locally stored matrix data in parallel. This approach dramatically reduces off-chip data transfers, leading to both faster computation and lower power consumption [16, 40, 59]. As a result, PIM systems offer substantial efficiency and performance gains in environments constrained by power or bandwidth, from edge devices to data centers.

PIM devices can be equipped with CPUs or GPUs as replacements for conventional memory [10, 38, 40], and recent real-world PIM devices illustrate varying trade-offs in programmability, compatibility, and system integration. UPMEM-PIM [10] integrates RISC-based processing units (PUs) into DRAM devices, enabling in-memory execution of entire applications. It uses a protocol compatible with the JEDEC DDR4 standard [19], allowing it to be deployed as a drop-in replacement for conventional DDR4 DIMMs. Although it occupies standard main memory slots, UPMEM-PIM cannot function as main memory as memory access from general user applications is restricted and entirely blocked during PIM execution.

In contrast, HBM-PIM [35, 40] and GDDR6-AiM [38] accelerate specific operations like GEMV rather than running entire applications. Thus, they must be tightly integrated with the processor and work in conjunction with it to complete end-to-end applications. HBM-PIM adheres to the standard HBM2 interface [21] and separates memory modes into normal and PIM modes to employ conventional memory commands (i.e., reads and writes) to trigger PIM

operations. However, regular memory requests from conventional workloads cannot be processed while in PIM mode, and switching between memory modes incurs non-negligible latency because it requires a predefined sequence of commands to change the mode. Consequently, conventional applications can suffer from significant memory access latency. GDDR6-AiM [38] extends the standard GDDR6 interface with new compute and data commands, enabling it to handle both standard memory and PIM requests without mode changes. However, this approach requires a dedicated memory subsystem for new commands or modifications to the processor-side memory controller (MC), which are specific to GDDR6-AiM and cannot be reused for other PIM architectures. As a result, adopting GDDR6-AiM necessitates PIM-specific customization of the memory interface, limiting its portability and general applicability.

While real-world PIM architectures have proven effective for accelerating memory-bound applications, their integration into general-purpose systems remains limited due to protocol constraints. First, these protocols do not support fine-grained interleaving of memory requests from the processor and PIM, requiring explicit mode switching or coarse-grained serialization. This becomes particularly problematic in systems where PIM must serve not only as a compute engine but also as general-purpose memory, such as mobile or edge platforms, and also leads to scheduling challenges in coordinating memory access between the processor and PIM. Prioritizing PIM execution excessively delays normal memory requests, whereas prioritizing host memory access can lower PIM throughput and violate Quality-of-Service (QoS) for PIM workloads. Second, compatibility remains a significant challenge. For example, GDDR6-AiM supports fine-grained interleaving but requires modifications to the memory subsystem, including the MC, to support new commands. Such incompatibility hinders adoption, as each PIM design requires custom host integration, increasing hardware complexity and preventing wide deployment across platforms. Therefore, a compatible PIM protocol must be designed to comply with existing DRAM standards, particularly considering the limited command/address (CA) space of commodity memory interfaces.

To address these challenges, we propose ComPASS, a compatible PIM protocol architecture and scheduling solution that enables efficient collaboration between the processor and PIM devices. Considering the limited CA space and the fact that most PIM architectures initiate execution via multi-bank activation (ACT), we introduce a new PIM command, PIM-ACT, which activates multiple banks and initiates a specific PIM operation simultaneously. It includes an optype field that allows each PIM device to define its own semantics, supporting architectural diversity while maintaining compatibility. To reduce command transmission overhead, we incorporate a PIM request generator in the MC, which autonomously generates PIM requests, alleviating the burden on processor cores. We further show that ComPASS can efficiently support PIM operations, which do not use multi-bank ACT, through architecture-aware PIM-ACT optimization, and can be applied to existing PIM devices such as HBM-PIM [40] and GDDR6-AiM [38]. Moreover, we propose two scheduling solutions, static and adaptive throughput balancers, that coordinate PIM and normal memory requests. These ensure that general-purpose workloads are served without excessive delay, while maintaining consistent performance for PIM execution. We evaluate ComPASS using microbenchmarks and GEMV operations from GPT-3 inference (6.7B and XL models [6]), representing typical PIM workloads. Our results show that ComPASS meets token generation latency targets where the processor runs various combinations of SPEC CPU 2017 workloads [56] and multi-threaded GAP Benchmark Suite (GAPBS [3]) workloads concurrently with PIM. The key contributions of this paper are as follows:

- We propose a new PIM command, PIM-ACT, and the memory subsystem with PIM request generator to establish a compatible PIM protocol that can be adapted to various PIM architectures.
- We introduce two scheduling solutions, the static and adaptive throughput balancers, to manage PIM and standard memory requests, ensuring memory access for general workloads while maintaining consistent PIM performance.
- Through simulations accelerating GEMV operations for GPT-3 inference workloads, we demonstrate that ComPASS meets token generation latency requirements, achieving 10.75× GEMV performance improvement over the system without PIM even when memory-intensive SPEC CPU workloads run together.

2 Background: Real-world PIM Devices

Recently, Processing-In-Memory (PIM) devices from DRAM manufacturers [10, 35, 38, 40] have proven viable in real-world systems. They leverage DRAM bank-level parallelism, employing a SIMD structure to access and process data from each bank in parallel with a single instruction.

UPMEM has commercially released UPMEM-PIM [10], the first programmable PIM device, implemented in the form of DDR4-2400 DIMMs [19]. Each DRAM chip of UPMEM-PIM contains a Data Processing Unit (DPU) per bank to enable parallel computations across DRAM banks and accelerate memory bandwidth-limited applications (Figure 1(a)). Each DPU is a RISC-based processor that operates with ISA composed of dozens of instructions. PIM instructions are stored in the DPU's IRAM and executed sequentially. Additionally, each DPU is equipped with WRAM, which serves as a scratchpad or cache, enabling efficient access to reusable data. Since UPMEM-PIM can perform various computations flexibly through DPUs, it can be considered a general-purpose PIM, and the address space of UPMEM-PIM is isolated from the main memory, so it functions similarly to an accelerator from the CPU's perspective.

Some PIM devices [35, 38, 40] focus on accelerating specific operations that dominate key workloads (i.e. domain-specific PIMs). Samsung's HBM-PIM [35, 40], designed for GEMV and element-wise operations, extends HBM2 by integrating one PU per two banks (Figure 1(b)). Each PU consists of sixteen FP16 FPUs and several register files GRF_A/B for vector data, SRF for scalars, CRF for PIM instructions, and control registers. HBM-PIM supports three modes–SB, HAB, and HAB-PIM–to distinguish PIM operations from standard memory access using JEDEC-compliant commands. In SB mode, each bank independently handles standard memory requests. In HAB and HAB-PIM modes, all PUs in a channel and their associated banks (even or odd banks) operate in parallel. HAB mode allows updating PU registers through DRAM write (WR) commands, while HAB-PIM mode enables concurrent bank access and PU execution using read (RD) or WR commands. Mode switching is triggered by a predefined sequence of memory commands sent to specific addresses. To initiate PIM execution, the host switches

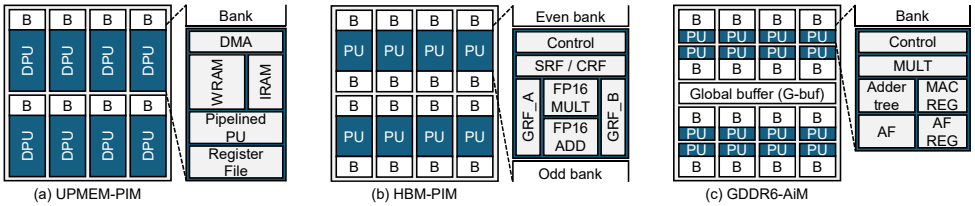


Figure 1: Existing real-world PIM architectures.

to HAB mode and writes instructions and data into the PU register files (CRF and control). Then, in HAB-PIM mode, the host updates GRF and issues DRAM column commands (RD/WR), where the address field is interpreted both as a DRAM address and as the operand index for GRF or SRF. To ensure correctness, the corresponding DRAM row must be activated (ACT) before issuing RD or WR commands in both HAB and HAB-PIM modes.

SK Hynix's GDDR6-AiM [38] accelerates GEMV and activation functions by embedding a PU in each GDDR6 memory bank (Figure 1(c)). Each PU includes MAC units, an adder tree, an activation unit (e.g., sigmoid), and local registers (MAC_REG and AF_REG) for intermediate results. A global buffer (G-buf), shared across all banks, holds input vector data reused between operations. GDDR6-AiM adds sixteen PIM commands beyond the JEDEC standard, categorized into three types: multi-bank ACT, data transfer, and compute. Like HBM-PIM, it executes PIM operations through the MC, which issues PIM commands along with operand addresses or indices. Execution begins with a multi-bank ACT to open rows across multiple banks, followed by data transfer or compute commands to perform GEMV or activation functions. If the operand data is located in a different row, all banks must be precharged, and new rows are activated in parallel through another multi-bank ACT.

3 Motivation

The rise of LLMs has driven interest in domain-specific PIM architectures to accelerate key operations, prompting efforts to integrate PIM into systems from data centers to mobile platforms [17, 39, 46, 54]. However, broad PIM adoption still faces two major challenges: protocol compatibility and memory scheduling. First, a general-purpose protocol is needed to integrate PIM without major modifications to existing DRAM standards. Without such a protocol, each PIM design requires separate changes to the memory subsystem, increasing integration costs. For example, GDDR6-AiM [38] introduces over a dozen custom commands, requiring a dedicated memory subsystem only for itself. HBM-PIM [40] adheres to HBM2 [21] standards but incurs latency overhead from mode transitions. A complete round-trip transition between PIM mode and normal mode takes approximately 350ns, during which memory access is entirely stalled. This overhead makes frequent mode switching highly inefficient. The LPDDR-PIM [28], still under development, proposes additional commands to ease mode transitions, but this approach is ineffective for architectures that do not rely on such mechanisms. Thus, a compatible protocol is essential to reduce system complexity and ensure interoperability across PIM designs.

Second, domain-specific PIMs require frequent coordination with the host processor, as they cannot independently execute full workloads. Also, since they often serve as main or device memory, PIM operations must coexist with normal memory traffic. However, PIM

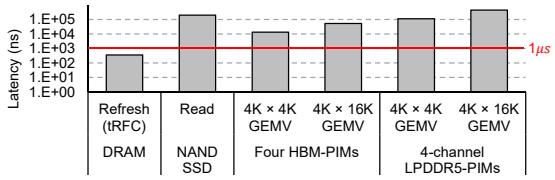


Figure 2: Comparison of PIM-based GEMV execution time, DRAM refresh, and NAND-based SSD read latency.

operations typically trigger multi-bank ACT, blocking normal memory access. For example, a 4096×4096 GEMV on four HBM-PIM modules takes $13\mu s$ [40] (Figure 2), during which normal requests stall. This delay worsens in mobile systems with limited PIM bandwidth; the same GEMV takes over $100\mu s$ on a 4-channel LPDDR-PIM [28], comparable to NAND SSD latency. Larger GEMVs, such as 4096×16384 in fully connected layers of LLM inference, exacerbate the delay. If normal requests are blocked across consecutive PIM operations, user-interactive tasks experience severe delays, degrading system responsiveness. Also, frequent switching between PIM and normal requests incurs bank-state transition overhead, while long uninterrupted bursts of either type can starve the other, violating QoS. Thus, a balanced scheduling policy is required to optimize memory bandwidth utilization while minimizing such disruptions.

Designing such a solution must also consider hardware constraints. The limited CA width defined by JEDEC standards restricts the number of additional PIM commands. While reserved-for-future-use (RFU) commands can be leveraged, they are scarce (e.g., DDR5 provides only four [22]). LPDDR5 and GDDR6 offer even less availability, lacking RFU commands [23, 24]. In these cases, repurposing don't-care bits in existing commands, such as NOP (No operation), may offer a partial solution—but the number of usable commands remains limited. Thus, the protocol must be designed to accommodate these constraints while maintaining broad compatibility across devices, enabling unified host-side implementation.

To address these challenges, we propose ComPASS, a compatible PIM protocol and scheduling solution for efficient PIM integration in modern systems.

4 Compatible PIM Protocol and System Support

4.1 PIM-ACT: A Memory Command for PIM

To establish a compatible protocol, we first analyze the common operational patterns across PIM devices. Many PIM architectures exploit bank-level parallelism by activating rows in multiple banks and accessing data concurrently during computation. For example, in GEMV, each bank activates a row containing part of the matrix, and each PU performs multiply-and-accumulate (MAC) operations using vector data in its register (e.g., GRF_A in HBM-PIM [40] or the global buffer in AiM [10]). For writing register values to DRAM,

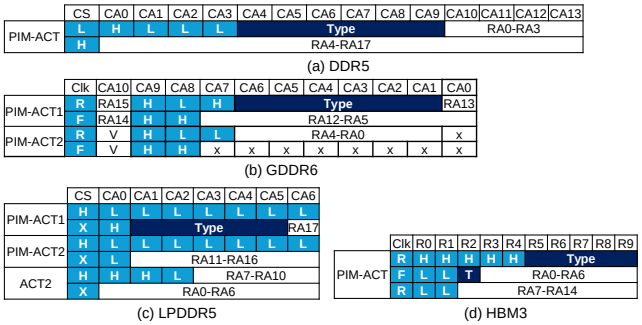


Figure 3: PIM-ACT formats in various DRAM standards. Each column represents an individual pin signal.

each PU activates a different row across the banks in parallel and stores the data to DRAM cells.

Based on this observation, ComPASS introduces a new memory command, PIM-ACT, which simultaneously activates rows in multiple banks and allows subsequent column commands (e.g., RD) to encode and trigger specific PIM operations. The MC issues PIM-ACT with a row address and a PIM operation type (optype), which defines the operation semantics. Following PIM-ACT, column commands are broadcast to multiple banks and interpreted according to the specified optype. Each PIM architecture can define its own optype set according to its operations, while still adhering to the common protocol semantics. This design allows different PIM devices to communicate with the host using the same PIM-ACT interface, ensuring compatibility.

PIM-ACT is implemented using RFU commands defined in various JEDEC standards (Figure 3). In DDR5 [22], one RFU command is used to support up to 64 distinct PIM operations. In LPDDR5 [23] and GDDR6 [24], which lack RFUs, we repurpose don't-care bits in the NOP command to encode PIM-ACT. To overcome the limited CA width, we adopt a multi-cycle command format, allowing both the row address and optype to be delivered over multiple cycles. LPDDR5 already divides ACT into ACT1 and ACT2 across two cycles, so we leverage and extend it as illustrated in Figure 3(b) and (c). This format requires additional decoding time, increasing tRCD (row-to-column delay) for PIM-ACT by one cycle.

PIM devices can be designed to recognize PIM-ACT and support up to 64 architecture-specific operations. For example, HBM-PIM [40] can leverage PIM-ACT to encode 35 commands that are actually used in practice. The optype field specifies both the operation and its operand sources and destination, offering detailed instruction semantics. Also, considering PIM requests according to data direction–write (host-to-PIM) vs. read (DRAM-to-PIM)–ComPASS combines this with an optype to express over 64 effective operations in a single PIM-ACT format.

When the MC receives a PIM read or write request, it issues PIM-ACT if the target row is not already activated. After tRCD cycle, the MC can issue a column command for the PIM request. If subsequent PIM requests share the same optype and row, they are treated as row hits and issued at tCCD_L (column-to-column delay within bank group) interval. Otherwise, if a request targets a different row, the MC must first precharge all banks using the standard PREA command.

4.2 System Support to Exploit PIM-ACT

To ensure the issuance of PIM commands through PIM-ACT, the MC must receive PIM read/write requests in the specified sequence. In this case, PIM requests must be generated at an extremely high frequency to fully utilize the maximum PIM bandwidth, which places a substantial load on host processors (i.e., CPU, GPU, or NPU) [35]. For example, in HBM-PIM, where read or write requests are utilized to perform PIM operations, up to 1,024 threads generate memory requests to achieve maximum PIM bandwidth. This approach leads to excessive host processor core usage during PIM operations, ultimately degrading overall system performance. To alleviate this burden, we propose to add a PIM request generator within the MC, which breaks down macro-level requests from the host into micro-level PIM requests (Figure 4(a)).

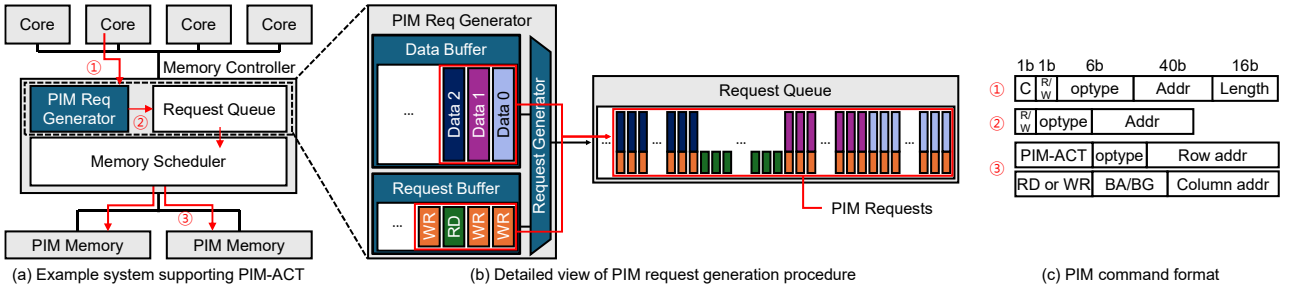
As shown in Figure 4(b), the PIM request generator consists of a request buffer to store macro-level requests, a data buffer that holds the data necessary for processing these requests, and a decoding logic. The core issues a macro request that defines the optype, starting address of the operand, and operation length, to the PIM request generator. Then, the PIM request generator decodes it and generates multiple micro PIM read or write requests to the MC's request queue. The MC schedules these micro PIM requests with regular memory requests and issues PIM-ACT and RD (or WR) commands to process them. While request generation is a general concept, ComPASS enables a single, unified generator to support diverse PIM devices by standardizing the macro interface, as further detailed in the following sections.

4.3 The Details of Macro and Micro Requests

Figure 4(c) shows the detailed structure of PIM macro and micro requests and the corresponding DRAM commands. A macro request comprises a 1-bit completion flag, a 1-bit RW indicator, a 6-bit optype, a 40-bit address, and a 16-bit length field, totaling 64 bits. This design allows the processor to issue macro requests that align with its cache-line size. The completion bit indicates whether all associated micro requests have been issued, while the RW indicator specifies the request type between read and write. The optype field defines the PIM operation, and the address field specifies the starting physical address for the macro request. The length field indicates the length of the operation, specifying how many read or write micro requests are needed to complete it. For PIM write requests, however, the length field is split to represent both the operation length and the offset of the data buffer address, as data must be supplied from the data buffer within the PIM request generator.

The 40-bit address field supports up to 1 TB of addressable memory space, sufficient for typical CPU or mobile system requirements. The address and length fields are adjustable because they are system-dependent. If the system's memory capacity increases, the address field can be extended, or the address can be modified to exclude the cache-line offset, thereby expanding the addressable capacity. Conversely, if 40 bits for the address are not required, the address field can be reduced, allowing more bits to the length field.

A micro request consists of the same fields as a macro request, except for the completion bit and length fields. Each micro request represents a PIM read or write and specifies the type of PIM operation to be performed. The MC schedules PIM micro requests with



(a) Example system supporting PIM-ACT (b) Detailed view of PIM request generation procedure (c) PIM command format

Figure 4: Memory system with PIM request generator, which contains request buffer and data buffer to store PIM macro requests and data for PIM operations, respectively. Request generator decodes PIM macro requests and generates PIM micro requests.

normal memory requests and issues the necessary memory commands to fulfill them. For PIM micro requests, however, PIM-ACT is used in place of the standard ACT command, with the optype passed along. Subsequent RD or WR commands behave like regular column commands. Consequently, this approach requires a separate command only for the process issuing ACT, minimizing the need for extensive changes to the memory controller logic.

4.4 Managing PIM Request Generator

Processors can access the PIM request generator's buffers via memory-mapped IO (MMIO). This access is managed by the OS kernel and PIM libraries as a privileged operation and protected as a critical section to ensure exclusive access from multiple cores. To initiate a task, the host processor writes macro requests and data to their respective buffers and then triggers the PIM request generator via MMIO. The completion status of macro commands can be monitored either through polling or an interrupt-driven method. In the polling approach, a designated core periodically checks the completion bit, though this may introduce additional overhead to the processor core. Alternatively, we leverage an interrupt-driven method where the MC sends an interrupt to the processor upon task completion. To prevent frequent interrupts, the PIM request generator triggers an interrupt only when the completion bits of all macro requests are set. Upon receiving the interrupt, the processor reads the completed PIM results to proceed with subsequent operations or sends new macro requests to initiate other PIM operations.

Modern computer systems are typically equipped with multiple MCs, each managing a distinct physical memory address range. In such systems, a dedicated PIM request generator is placed on each MC, generating micro requests for its local memory space. The host processor sends macro requests to each MC's generator individually and treats the PIM operation as complete only after receiving completion interrupts from all involved generators.

To support this coordination, the PIM vendor's software stack is expected to abstract multi-MC management by generating appropriate macro operations from user-level requests, synchronizing execution, and collecting completion signals from all PIM request generators, possibly with custom address mapping. Alternatively, since ComPASS is a protocol-level solution compatible with various PIM devices, the OS can provide a common runtime support (e.g., PIM libraries) to route macro requests to multiple PIM request generators and synchronize them, excluding vendor-specific tasks such as address mapping and macro request generation.

4.5 Architecture-Aware Optimization of PIM-ACT

ComPASS enables architecture-aware protocol optimizations (AAO) by flexibly leveraging PIM-ACT and optype support. Depending on the characteristics of the target PIM device, ComPASS allows tuning of DRAM timing parameters and bank ACT granularity to maximize execution efficiency. This tuning is enabled through information provided by the Serial Presence Detect (SPD [20]), which allows the MC to adapt behavior based on device-specific characteristics. By enabling AAO, ComPASS supports architectural optimizations without losing protocol compatibility or adding host-side complexity. Note, however, this assumes a single PIM device type per host processor, with customization applied accordingly.

First, some PIM operations do not involve DRAM row ACT. For example, commands that transfer data from the processor to PIM internal registers or buffers, or move data between PUs, do not access DRAM cells and therefore do not require ACT. For such cases, using the standard DRAM row timings–such as tRCD, tRAS (row active time), and tRP (row precharge time)–is unnecessarily conservative for PIM-ACT. Instead of introducing new command types, ComPASS supports per-optype timing customization, allowing the MC to apply tighter row timings based on optype. These per-optype row parameters are provided through SPD and loaded by the MC at boot time, enabling efficient execution without requiring changes to the overall protocol.

Second, for PIM architectures that activate rows at finer granularity (e.g., bank-level or bank-group-level ACT [27, 30, 31, 34, 60]), maintaining a common open row across all banks is unnecessary. In these cases, the SPD can specify whether each optype applies to all banks or a subset. This allows the MC to manage bank states independently, reducing unnecessary PREA and PIM-ACT overhead. However, a portion of the optype bits must be reserved to specify the target bank, reducing the available encoding space for PIM operations. One approach to address this limitation is to extend PIM-ACT with additional cycles, utilizing the multi-cycle command described in Section 4.1. For example, adding one extra cycle to the command (e.g., in an HBM3 system) provides 8 additional bits for optype, which can encode both bank IDs and compute semantics. Although this appears to increase tRCD, it can be resolved by specifying the target bank in the optype portion of the initial cycles and encoding the PIM operation in the final cycle.

Third, for compound operations where multiple computations are executed sequentially using a single DRAM column command

(e.g., PIM_ADD_ReLU triggered by RD), column-level timing constraints—such as tCCD_L, which governs the minimum delay between consecutive column commands—must be carefully adjusted to avoid violating timing or introducing stalls. In such cases, ComPASS supports per-optype column timing adjustments, again using parameters defined in SPD.

4.6 Address Space Management for PIM

A macro request generates a specified number of micro requests that begin at a specific physical address, striding by memory access size units. Therefore, data for PIM operations must be stored in a physically contiguous memory. To ensure this, we use HugeTLB [9] feature to allocate PIM data on huge pages (2MB or 1GB), depending on the required data size. Prior studies [45, 50, 63] also adopt huge pages to maintain contiguous memory allocation for PIM. HugeTLB dynamically allocates huge pages even when PIM operations run alongside regular applications, preventing PIM data from being swapped out. Additionally, when memory fragmentation prevents direct huge page allocation, the memory compaction feature [13] can be leveraged. While compaction introduces overhead, PIM target applications like LLM inference repeatedly access preallocated memory over long inference sequences, amortizing the initial allocation cost. However, for shorter PIM workloads, the overhead may become more pronounced, and potential system-wide impacts, such as performance interference with co-running applications, remain important considerations for future quantitative analysis.

4.7 Applying Proposed PIM Protocol to Existing PIM Architectures

To demonstrate the compatibility of ComPASS, this section illustrates how it can be applied to commercially available PIM architectures, unifying them under a single protocol without requiring major design changes. Table 1 shows how ComPASS applies to HBM-PIM [40] and GDDR6-AiM [38]. Optypes 0-31 are assigned to computation operations, and 32-63 to data transfer operations, simplifying decoding logic. HBM-PIM and GDDR6-AiM use 35 and 32 optypes, respectively. The same operation can be classified into different optypes depending on its source and destination.

In HBM-PIM, MUL, ADD, and MAD use eight optypes each, while MAC uses two. These commands involve reading data from the DRAM banks, so they are treated as read requests. In contrast, ReLU uses four optypes, operating on GRF data within the PU and writing results back to DRAM, making it a write request. MOV command uses five optypes based on the source and destination. For transfers between banks and GRFs, the request type (read or write) also determines the source and destination. When transferring data from the host processor (i.e., data buffer) to the register file (RF) within the PU, the address field specifies the type of RF (i.e., GRF, SRF, or CRF). Since these accesses do not involve DRAM row ACT, they are handled using PIM-ACT commands with row timing adjusted by AAO. Although ReLU and MOV are write requests, they transfer data internally between banks and PUs, and the PU ignores data from the host.

In GDDR6-AiM, although operations are more granular, fewer source-destination pairs reduce required optypes. Since GDDR6-AiM supports four-bank MAC (MAC4B) and all-bank MAC (MACAB), MAC operations are encoded using five optypes with AAO: four for MAC4B and one for MACAB. EWMUL also uses five optypes, as it can operate on each bank group independently or across all bank groups. AF (activation function) accesses only internal registers, making it suitable for row timing AAO. For RDCP and WRCP, which move data between G-buf and a specific bank, sixteen optypes with AAO are used to distinguish among the 16 banks. Other operations use a single optype each, with the data transfer direction indicated by the read/write request type. Like HBM-PIM, transfers between the host and internal registers (MAC_REG, AF_REG, and G-buf) benefit from row timing AAO. If a write request does not require host data, the PU simply ignores the transferred data.

Figure 5 presents an example PIM request generator API and pseudo-codes for executing GEMV on HBM-PIM and GDDR6-AiM with the PIM request generator. The operand matrix is horizontally partitioned across the DRAM banks. As shown in Figure 5(a), the functions **send_PIM_req()** and **send_PIM_data()** send requests and data to the memory-mapped request buffer and data buffer within the PIM request generator, respectively, while the function **run_PIM_request_generator()** initiates the PIM request generator to transmit micro-requests to the MC request queue. This process is common for both HBM-PIM and GDDR6-AiM.

For HBM-PIM (Figure 5(b)), the GRF holds only a segment of the input vector (sub-vector) due to the size constraints. Thus, the GRF must be updated with another sub-vector upon completion of each operation. The overall GEMV operation is performed in the following sequence. 1) The processor writes the entire input vector to the data buffer of the PIM request generator. 2) For each sub-vector, the processor sends macro PIM write and read requests indicating writing a sub-vector to GRF and processing the MAC operation, respectively. 3) Then, it sends macro write requests to store the results back in the DRAM banks. 4) Finally, it triggers the PIM request generator to generate micro PIM requests.

Figure 5(c) illustrates the pseudo-code for GEMV operations in GDDR6-AiM. The overall operation flow is identical to that of HBM-PIM, but two key differences exist: the input vector is held in the larger G-buf instead of the GRF, and each output is read back by the host processor rather than being stored in the DRAM bank.

5 Memory Scheduling with PIM-ACT Protocol

The proposed protocol introduces new memory request types, PIM read and PIM write, along with a new DRAM command, PIM-ACT. This addition necessitates a memory scheduler that can balance PIM and non-PIM requests. Since PIM workloads are offloaded to acceleration, ensuring sufficient PIM throughput to meet their QoS is the top priority. While invoking PIM operations implies the user's intent to allocate bandwidth for in-memory computation, fully stalling non-PIM applications can cause system-level issues. Therefore, the scheduler should allow non-PIM requests to proceed without excessive delays. To achieve this balance, ComPASS proposes scheduling strategies that prioritize PIM execution when needed while allowing normal memory requests to proceed without excessive delays. This section identifies key challenges in handling mixed PIM and non-PIM memory access, analyzes the limitations of prior scheduling policies, and proposes a new memory scheduling solution for PIM-attached systems.

Table 1: Examples of applying the proposed protocol to HBM-PIM and GDDR6-AiM architectures.

Architecture	Operation	SRC0	SRC1	DST	R/W	optype	address	length	AAO
HBM-PIM	MUL	GRF_A/B, SRF	Even/Odd banks	GRF_A/B	R	0-7	address	length	-
	ADD	GRF_A/B, SRF	Even/Odd banks	GRF_A/B		8-15	address	length	
	MAD	GRF_A/B, SRF	Even/Odd banks	GRF_A/B		16-23	address	length	
	MAC	GRF_A	Even/Odd banks	GRF_B		24-25	address	length	
	ReLU	GRF_A/B	-	Even/Odd banks	W	26-29	address	length | offset	
	MOV	Host processor	-	RF	W	32	address	length | offset	row timing
		Even/Odd banks		GRF_A/B	R	36-39	address	length	-
		GRF_A/B		Even/Odd banks	W			length | offset	
GDDR6-AiM	MAC	G-buf	4/16 banks	MAC_REG	R	0-4	address	length	ACT granularity
	EWMUL	2 banks per bankgroup		1 bank per bankgroup		8-12	address	length	
	AF	MAC_REG	-	AF_REG		13	address	length	row timing
	RDCP	1 bank	-	G-buf	R	32-47	address	length	ACT granularity
	WRCP	G-buf		1 bank	W			length | offset	
	WRGB	Host processor	-	G-buf	W	48	address	length | offset	row timing
	RDMAC	MAC_REG	-	Host processor	R	49	address	length	
	WRBIAS	Host processor	-	MAC_REG	W	50	address	length | offset	
	RDAF	AF_REG	-	Host processor	R	51	address	length	
	WRBK	Host processor	-	16 banks	W	52	address	length | offset	-

```
# send_PIM_req(RD/WR, optype, addr, length|offset) : sends macro req to req buf
# send_PIM_data(src addr, data buf addr, length)   : sends src data to data buf
# run_PIM_request_generator()                      : initiates PIM req generation
# tile       : the matrix tile corresponding to the output produced
#              by the parallel computation of all PUs
# sub-vec    : part of vector corresponding to GRF or G-buf size
```

(a) PIM request generator API

```
01 send_PIM_data(vector addr, data buf offset, vector length)
02 for each tile in matrix
03   for each sub-vec
04     send_PIM_req(WR, MOV[data buf to GRF_A], GRF addr, sub-vec length|offset)
05     send_PIM_req(RD, MAC[GRF_B=GRF_A×Bank], tile addr, tile length)
06   send_PIM_req(WR, MOV[GRF to Bank], result addr, result length|offset)
07 run_PIM_request_generator()
```

(b) HBM-PIM

```
01 send_PIM_data(vector addr, data buf offset, vector length)
02 for each tile in matrix
03   send_PIM_req(WR, WRBIAS, MAC_REG addr, bias length|offset)
04   for each sub-vec
05     send_PIM_req(WR, WRGB, G-buf addr, G-buf length|offset)
06     send_PIM_req(RD, MACAB, tile addr, G-buf length)
07   send_PIM_req(RD, RDMAC, MAC_REG addr, 1)
08 run_PIM_request_generator()
```

(c) GDDR6-AiM

Figure 5: Pseudo code of GEMV execution on HBM-PIM and GDDR6-AiM with PIM request generator.

5.1 Bank State Transitions for PIM Requests

PIM requests are accompanied by PIM-ACT, which changes the state of all banks within a channel. Therefore, the MC should consider the following scenarios: 1) After processing PIM requests, a PREA must be issued before non-PIM requests. 2) To initiate a PIM request following non-PIM requests, PREA and PIM-ACT must be issued. 3) Successive PIM requests on the same row are treated as row hits, but those targeting different rows or with a different optype require PREA and PIM-ACT again. Since PREA is required to synchronize bank states between PIM and non-PIM requests, frequent switching should be avoided. Additionally, because PIM requests require PREA when the target row changes, it is beneficial to process non-PIM requests before issuing PIM-ACT again.

5.2 Limitation of Existing Scheduling Solutions

Prior scheduling policies like FR-FCFS [51] can be adapted to manage PIM and non-PIM requests. Figure 6 illustrates how various scheduling policies process given request sequences. FR-FCFS minimizes unnecessary PREA and PIM-ACT commands by prioritizing row-hit requests after issuing PIM-ACT. However, if memory-intensive non-PIM workloads generate many row-hit requests, PIM execution is delayed significantly (Figure 6(a)). Conversely, non-PIM requests may also suffer in FR-FCFS. Since non-PIM requests can be processed with bank interleaving, handling multiple requests simultaneously is advantageous. However, in FR-FCFS, non-PIM requests are not guaranteed to be processed consecutively, as a single preceding PIM request, which takes priority, delays all subsequent non-PIM requests that cause row conflicts until all PIM requests for that row are completely processed (Figure 6(b)). Using batch schedulers like PAR-BS [44] improves balance by scheduling within request batches, but they can fragment row-hit PIM requests across batches. This leads to excessive PREA commands and degrades memory performance (Figure 6).

To schedule bandwidth-heavy PIM workloads, Thread Cluster Memory (TCM [36]) scheduler can be used, as it clusters threads into latency- and bandwidth-sensitive clusters. It rotates memory access priority among bandwidth-sensitive threads, helping distribute bandwidth across threads, but limiting PIM bandwidth when multiple memory-intensive CPU threads are present. Also, if the request queue is insufficient or the priority rotation interval is too short, consecutive PIM requests to the same row can be fragmented, increasing PREA and PIM-ACT overhead. With latency-sensitive threads, PIM requests are always deprioritized, and even a single interleaved non-PIM request can trigger PREA during a PIM burst. To address these issues, we gradually propose two memory scheduling solutions, static and adaptive throughput balancer.

5.3 Static Throughput Balancer

Static throughput balancer (ST-BLC) limits the number of consecutive requests of each type (PIM or non-PIM) to prevent either from monopolizing memory bandwidth. The MC assigns the highest priority to a specific type of request for a fixed number of accesses–P for PIM and N for non-PIM–and schedules requests within the type using FR-FCFS. Priority switches to the other type when: (1) the number of consecutive requests of one type reaches the threshold, (2) there are no more requests of the prioritized type, or (3) for PIM,

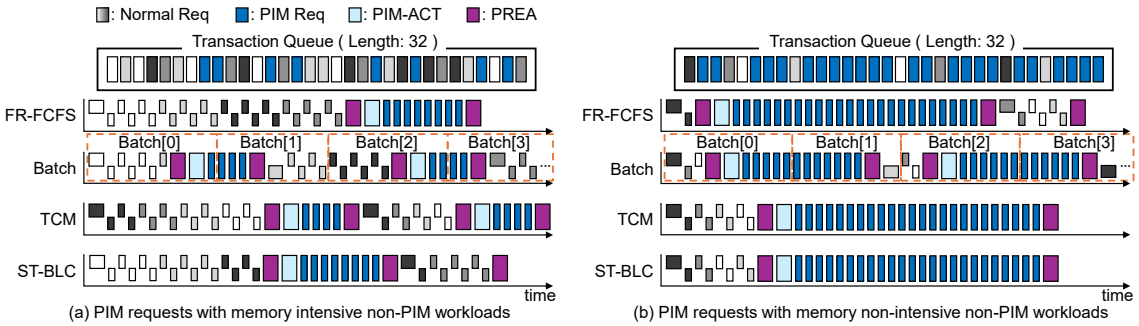


Figure 6: Scheduling policy comparison under PIM/non-PIM co-execution. Normal requests from different threads are shown in different shades of gray. PIM requests, PIM-ACT, and PREA commands are shown in blue, light blue, and purple, respectively.

no row-hit remains and a PREA is required. This approach helps reduce PREA overhead by grouping requests of the same type up to a defined threshold (Figure 6).

ST-BLC must select appropriate values for P and N to achieve the desired performance. The value of P directly impacts both PIM throughput and non-PIM latency. Larger P improves PIM throughput but can delay non-PIM requests. Since a PREA is required whenever the target row changes, there is no benefit in setting P larger than the number of columns in a DRAM row. Also, power-of-two aligned P values (e.g., 16, 32) are preferred when P is smaller than the full row. For example, in LPDDR5, a row contains 64 columns, allowing up to 64 PIM requests per row. If P is set to 48, the MC will issue a PREA after 48 PIM requests, serve N non-PIM requests, and then re-issue another PREA to process the remaining 16 PIM requests–resulting in two PREA operations for a single row. N is less constrained and can be tuned based on the desired trade-off between PIM and non-PIM performance; a higher value of N provides more throughput to non-PIM requests. However, setting both P and N too low leads to frequent switching and increased PREA overhead, reducing overall memory efficiency. Given these considerations, we present a sensitivity analysis of P and N in Section 7.

5.4 Adaptive Throughput Balancer

Selecting an optimal N for the ST-BLC requires considering multiple factors based on the characteristics of running workloads. A small N ensures high PIM performance; but if PIM bandwidth demand is low, it unnecessarily delays conventional workloads. Conversely, a large N can hinder PIM throughput under memory-intensive non-PIM workloads. Thus, a static approach may either fail to provide stable performance for PIM workloads or hurt system efficiency. To address this, we propose adaptive throughput balancer (AT-BLC), which adjusts N dynamically at runtime based on PIM execution progress. It ensures PIM QoS and allocates the remaining bandwidth to conventional workloads.

AT-BLC monitors the number of PIM micro-requests issued (W_{cur}) and the elapsed cycles (T_{cur}) since the start of the PIM operation. At every refresh interval, it adjusts N by a predefined delta (d)–increasing N when PIM progress exceeds the target and decreasing it when progress falls behind. To enable this adjustment, the target completion time (T) and total number of micro-requests (W) must be initialized before the PIM operation begins (prior to the pseudo-code in Figure 5). T and W values can be derived from the size of the PIM operation, and these calculated values are passed to the MC during initialization. The progress of PIM operation is evaluated based on the following two inequalities.

$$\frac{(W_{\text{cur}}/T_{\text{cur}} - W/T')}{W/T'} \leq -\lambda \quad (1)$$

$$\frac{(W_{\text{cur}}/T_{\text{cur}} - W/T')}{W/T'} \geq \lambda \quad (2)$$

T' is the margin-adjusted threshold (i.e., 90% of T), and λ defines the sensitivity for adjusting N. If Inequality (1) is satisfied, PIM execution has lagged behind, so AT-BLC decreases N by d to prioritize PIM. On the other hand, if Inequality (2) is satisfied, indicating sufficient PIM progress, AT-BLC increases N by d to give non-PIM requests more scheduling opportunities. However, to prevent over-expansion of N under memory non-intensive processor workloads, the MC only increases N only if more than 50% of the allocated N slots have been actually consumed on average.

In FR-FCFS and batch scheduling, the arrival time of requests in the queue affects scheduling, requiring both regular memory requests and PIM requests to be managed within a single queue. However, ST-BLC and AT-BLC can use separate queues for PIM and non-PIM requests, as they prioritize either PIM or non-PIM requests at the top level. Thus, we add an additional queue dedicated solely to PIM requests for these policies. Since PIM requests are issued sequentially by the PIM request generator, and no further reordering is required between them, this queue only has two entries, incurring negligible area cost.

6 Experimental Methodology

We evaluate our proposed protocol with PIM-ACT and scheduling policies using three types of memory simulators. To verify the protocol's functionality and impact on PIM performance in HBM-PIM, we implemented PIM-ACT in PIMSimulator [53], which models both the functionality and timing of HBM-PIM [40]. We tested four microbenchmarks, GEMV (4K×4K), ADD (1M vectors), MUL (2M vectors), and ReLU (4M vector), using the kernel tests provided by PIMSimulator. Additionally, we implemented PIM-ACT in AiM simulator [14, 43], which models the timing of GDDR6-AiM, to assess its impact on GDDR6-AiM performance, evaluating GEMV (4K×4K) and MUL (2M vectors) microbenchmarks.

We also modified Ramulator [37] to evaluate the impact of ComPASS on the performance of both PIM and host processor workloads, assessing overall system performance under different scheduling policies. Since Ramulator provides only simplistic out-of-order CPU

Table 2: System configuration for simulation.

Processor configuration	
CPU	16 O3 cores @ 2.4 GHz clock frequency 8 MB shared LLC
MC	4 memory channels 64-entry request queue / channel 2-entry PIM request queue / channel Interleaving (Row/BA/BG/RK/Col/CH)
PIM Req Gen	32-entry request buffer / channel 4 KB data buffer / channel
PIM configuration	
Common	4-channel 2-rank LPDDR5-6400-based PIM 2 KB row size, 32 B access granularity
LPDDR-PIM	HBM-PIM-based architecture 1 PU / 2 banks, operating @ tCCD_L GRF_A (512 B), GRF_B (512 B) CRF (256 B), SRF (64 B)
LPDDR-AiM	GDDR6-AiM-based architecture 1 PU / 1 bank, operating @ tCCD_L Shared global buffer (2 KB) MAC_REG (2 B), AF_REG (2 B)

Table 3: PIM target LLM models and GEMV configurations.

Model	Multi-Head Attention			FC
	QKV gen	Attention & Context	Projection	
GPT-3 6.7B	(4K×4K)×3	(128×4K)×32 (4K×128)×32	4K×4K	4K×16K 16K×4K
GPT-3 XL	(2K×2K)×3	(128×4K)×16 (4K×128)×16	2K×2K	2K×8K 8K×2K

core models, we assume a CPU-based system with PIM devices (Table 2). Based on the increasing adoption of LPDDR5 in modern systems [26, 28, 33], we modeled the processor with sixteen cores, 8MB LLC, and four memory channels equipped with LPDDR5-6400-based 2-rank PIM devices [23]. Each memory channel is controlled by an independent MC with a 64-entry request queue and 2-entry PIM request queue. The memory system uses interleaving across all channels, ranks, bank groups, and banks, following the bit mapping order from MSB to LSB as: row, bank, bank group, rank, column, and channel. PIM request generators in each MC have a 32-entry request buffer and a 4 KB data buffer. For PIM devices, we extended LPDDR5 memory, which can be utilized for the main memory of CPU system, to include PIM functionality, creating LPDDR-PIM and LPDDR-AiM that follow the designs of HBM-PIM [40] and GDDR6-AiM [38], respectively. Also, we implemented five scheduling policies: **FR-FCFS** [51], **Batch** [44], **TCM [36]**, **ST-BLC**, and **AT-BLC**. The batch size of **Batch** and the values of P and N for **ST-BLC** are all set to a default of 64, while the delta (d) and lambda (λ) values for **AT-BLC** are set to defaults of 8 and 0.01, respectively, adjusting N from 16 to 128. For **TCM**, we statically clustered threads based on MPKI with a `clusterThresh` of 8%. PIM workloads, which require high bandwidth and exhibit strong row buffer locality, were treated as bandwidth-sensitive with the lowest niceness. To reduce PIM request fragmentation, we set the `shuffleInterval` to 500 cycles.

Table 4: CPU workloads composition.

	Composition	BW Util. (GB/s)	Req Q Util.
CPU-hi1	[519, 510, 505, 549] × 3	24.83	57.31%
CPU-hi2	[510, 505, 510, 519] × 3	26.22	55.68%
CPU-hi3	[505, 505, 549, 519] × 3	25.17	63.54%
CPU-low1	511, 525, 500, 541	1.11	0.44%
CPU-low2	557, 523, 538, 507	4.30	4.47%
CPU-low3	507, 502, 544, 520	6.39	7.24%
BFS	16 threads	23.9	98.71%
PR	16 threads	25.6	98.48%

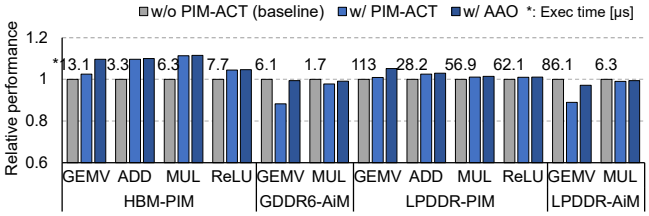


Figure 7: PIM microbenchmarks performance in HBM-PIM, GDDR6-AiM, LPDDR-PIM and LPDDR-AiM with PIM-ACT.

The workloads used in our evaluations are categorized into PIM and CPU workloads. For PIM workloads, we use GEMV with varying matrix sizes derived from different GPT-3 FP8 inference models, GPT-3 6.7B and GPT-3 XL [6] (Table 3). We assume that the GEMV operations of each model are offloaded to the PIM devices; for multi-head attention, we assume that GEMV operations across multiple heads can be computed on PIM as a single large GEMV operation. Also, to account for the variation in matrix size as the sequence progresses, we conservatively evaluated performance using large matrices after sufficient key-value cache buildup. We also set a target QoS requirement, where generating one token for each model must be completed within 50 ms [42, 47]. Based on related studies analyzing the breakdown of LLM inference execution time [28, 49], we assume that GEMV operations must be completed within approximately 83% of the total execution time. For CPU workloads, we constructed multi-programmed workloads by mixing SPEC CPU2017 benchmark applications [56] and multi-threaded GAP Benchmark Suite (GAPBS [3]) workloads. Table 4 presents the composition of each workload, their memory bandwidth utilization, and their memory request queue utilization; higher request queue utilization indicates more requests waiting due to queuing. For SPEC CPU2017 workloads, we composed CPU-hi1, hi2, and hi3, each consisting of twelve memory-intensive applications, and CPU-low1, low2, and low3, each consisting of four memory non-intensive applications. For GAPBS workloads, we used 16-threaded Breadth-First Search (BFS) and PageRank (PR) with a Kronecker graph.

7 Evaluation

7.1 PIM-Only Evaluation

Using PIMSimulator and AiM Simulator, we verified that PIM-ACT works correctly on both HBM-PIM and GDDR6-AiM. For HBM-PIM, PIM-ACT produces computational results identical to those of the original protocol. For GDDR6-AiM, trace analysis confirms that our protocol generates the same memory command sequence as the original protocol. Beyond verifying functional correctness,

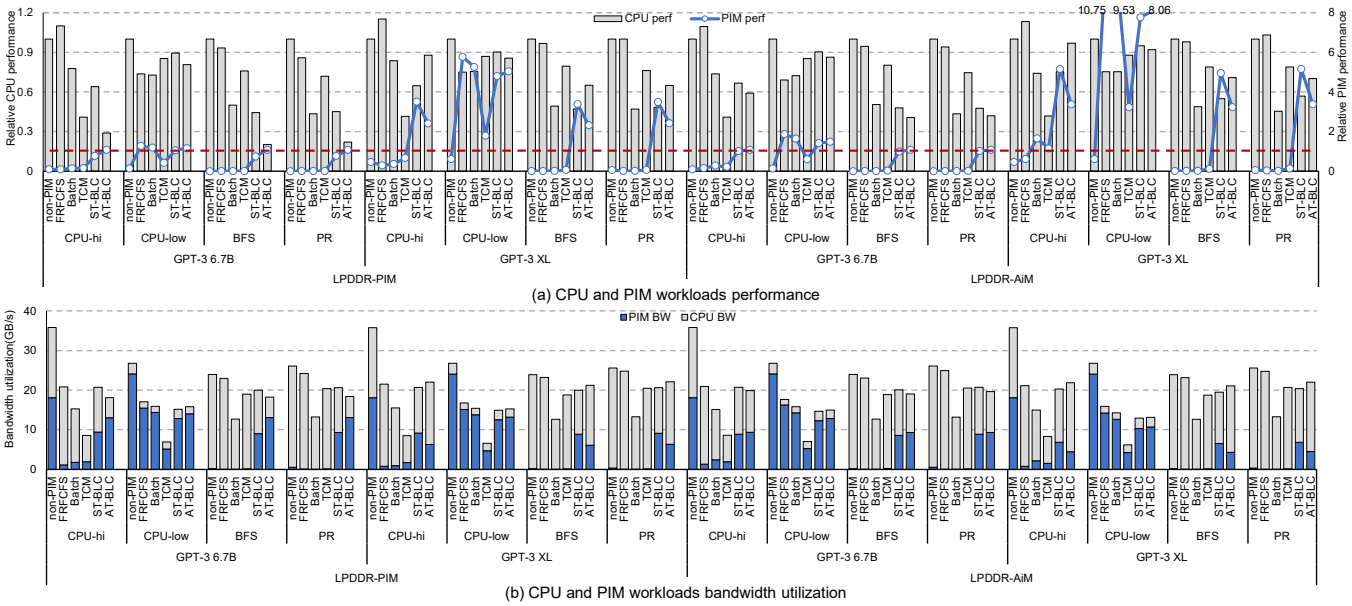


Figure 8: (a) Performance and (b) bandwidth utilization of PIM and CPU workloads in LPDDR-PIM and LPDDR-AiM systems with ComPASS under different scheduling policies. FR-FCFS, Batch, TCM, ST-BLC, and AT-BLC execute GPT-3 GEMV on PIM, while non-PIM runs them on the host processor. PIM performance is the inverse of execution time, normalized against the target execution time required to meet the QoS (the red dotted line).

we further evaluated the performance impact of PIM-ACT on both architectural models, HBM-PIM and GDDR6-AiM, as well as their LPDDR-based variants, LPDDR-PIM and LPDDR-AiM. In HBM-PIM and LPDDR-PIM, PIM-ACT improves execution efficiency by eliminating mode-switching request sequences, even without the AAO technique (Figure 7). In a system with four HBM-PIM modules, PIM-ACT achieves performance gains of 2.54%, 9.68%, 11.38%, and 4.51% over the baseline protocol on GEMV, ADD, MUL, and ReLU, respectively. The longer execution time of LPDDR-PIM reduces the relative impact of mode-switching overhead, leading to smaller performance gains (up to 2.52%). In contrast, using PIM-ACT without AAO in a 32-channel GDDR6-AiM system results in performance drops of 11.69% for GEMV and 2.19% for MUL. A similar trend was observed on the LPDDR-AiM, where performance decreased by 11.00% for GEMV and 0.89% for MUL. This degradation is due to the overhead of translating GDDR6-AiM's custom commands to PIM-ACT, which introduces unnecessary ACT latency for operations that do not involve actual DRAM ACT (e.g., write global buffer). However, applying AAO significantly mitigates this issue and effectively improves PIM performance. With AAO, PIM-ACT improves the performance of HBM-PIM to 9.65%, 10.00%, 11.53%, and 4.65% for GEMV, ADD, MUL, and ReLU, respectively, while achieving gains of 5.20%, 2.99%, 1.46%, and 1.13% on LPDDR-PIM for the same operations. In GDDR6-AiM, the GEMV performance drop is reduced from 11.69% to 0.59%, and from 2.19% to 0.85% for MUL. In LPDDR-AiM, GEMV performance drop decreases from 11.00% to 2.84%, though some loss remains due to LPDDR5's longer timing constraints (e.g., write-to-precharge latency).

These results show that despite using a unified command interface, PIM-ACT maintains performance comparable to, or even better than, device-specific protocols. Thus, this demonstrates that

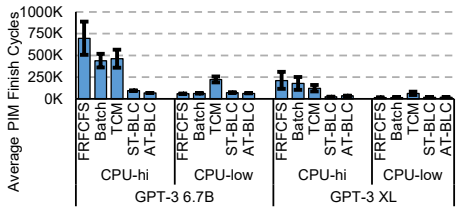


Figure 9: Mean and standard deviation of PIM operation (GEMV) execution times across MCs under different schedulers on LPDDR-PIM.

ComPASS can achieve functional compatibility and host-side integration without sacrificing efficiency.

7.2 CPU-PIM Co-execution Evaluation

We evaluated various scheduling policies, including **FR-FCFS**, **Batch**, **TCM**, **ST-BLC**, and **AT-BLC**, by measuring the performance and bandwidth utilization of co-executed PIM and CPU workloads on LPDDR-PIM and LPDDR-AiM systems with AAO (Figure 8). The GEMV operations from GPT-3 6.7B and GPT-3 XL are executed either on PIM devices or the host (**non-PIM**) alongside multiple CPU workloads. PIM performance represents the inverse of execution time, normalized to the target execution time for QoS (the red dotted line in Figure 8(a)). CPU performance indicates the weighted speedup normalized to the **non-PIM** baseline. CPU-hi and CPU-low indicate the average performance of CPU-hi1, -hi2, and -hi3 workloads and CPU-low1, -low2, and -low3 workloads, respectively.

Both **ST-BLC** and **AT-BLC** improve PIM performance in most cases compared to **non-PIM**, **FR-FCFS**, and **Batch** by eliminating inefficiencies in scheduling (Figure 8(a)). Even under memory-intensive CPU-hi workloads, **ST-BLC** and **AT-BLC** achieve 7.76×

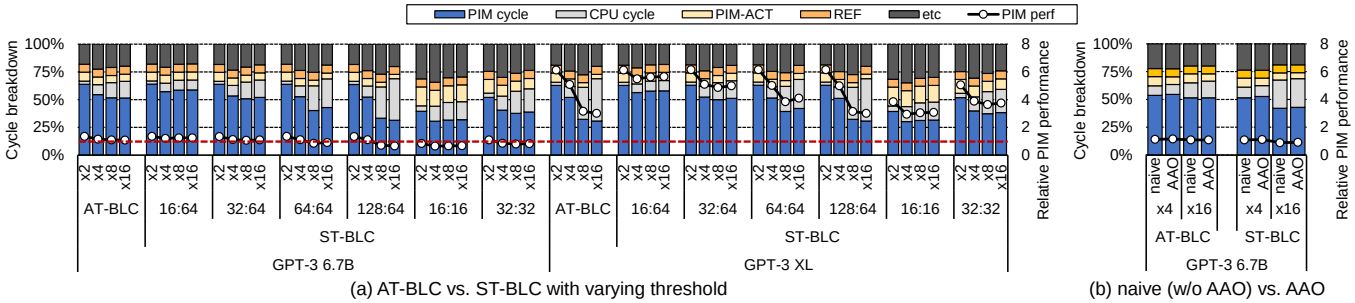


Figure 10: Memory cycle breakdown and PIM performance on LPDDR-PIM with (a) AT-BLC and ST-BLCs with varying threshold values; the threshold values for *N* and *P* of ST-BLC are indicated on the x-axis as *N*:*P*, and (b) without AAO (naive), under different CPU program counts (×*N*). PIM performance is normalized to the target QoS (the red dotted line).

and 10.75× gains on LPDDR-PIM, and 10.25× and 10.82× on LPDDR-AiM for GPT3-6.7B compared to **non-PIM**, respectively. Similar improvements are observed for BFS and PR, which exhibit comparable memory intensity. However, **ST-BLC**'s fixed threshold leads to a failure to meet PIM QoS in some cases–for example, on LPDDR-PIM when GPT-3 6.7B is executed alongside memory-intensive CPU workloads such as CPU-hi, BFS, and PR. Also, in other cases, such as with GPT-3 XL, it over-prioritizes PIM workloads, unnecessarily degrading CPU performance. In contrast, **AT-BLC** consistently satisfies PIM QoS across all cases. Moreover, once the QoS is satisfied, it dynamically allocates more bandwidth to CPU to improve CPU performance (Figure 8(b)). For instance, on LPDDR-PIM with CPU-hi and GPT-3 6.7B, **AT-BLC** assigns the least CPU bandwidth to preserve PIM QoS. However, with lighter PIM loads like GPT-3 XL, **AT-BLC** allows higher memory bandwidth for CPU workloads than **ST-BLC**, resulting in better overall CPU performance.

Unlike **AT-BLC** and **ST-BLC**, which use a separated PIM request queue, **non-PIM**, **FR-FCFS**, **Batch** and **TCM** struggle to insert PIM requests when CPU workloads like BFS and PR (Table 4) heavily occupy the queue. This results in low PIM bandwidth utilization and failure to meet the QoS. The issue is particularly severe in BFS, where **non-PIM** shows extremely low performance when executed alongside GPT-3 6.7B, with up to a 1,064× performance gap compared to **AT-BLC** with LPDDR-PIM. Even when there is room for PIM requests in the queue, they fail to meet the QoS under CPU-hi, regardless of the GPT model. **FR-FCFS** prioritizes CPU memory access due to bank interleaving, delaying PIM requests. **Batch** lacks sufficient PIM requests per batch, limiting the consecutive PIM execution after PIM-ACT. In **TCM**, when PIM workloads are co-scheduled with latency-sensitive applications (e.g., CPU-low), PIM requests are deprioritized, significantly reducing PIM throughput. With bandwidth-sensitive applications (e.g., CPU-hi), the time window during which PIM requests are given priority becomes limited, degrading PIM performance. Accumulated PIM requests also hinder efficient memory scheduling for other threads, ultimately lowering CPU performance. Although **non-PIM** shows high PIM bandwidth utilization on CPU-hi, it cannot leverage PIM internal bandwidth, resulting in low PIM workload performance.

In scenarios where the model size is small, requiring fewer PIM operations to meet the QoS demands, and CPU workloads are not memory-intensive (e.g., GPT-3 XL with CPU-low), all scheduling methods can satisfy the PIM workload QoS requirements. However, with the trend of increasing model sizes for higher performance, traditional scheduling methods are insufficient to satisfy PIM performance, underscoring the necessity of **AT-BLC**.

We further observe that **ST-BLC** and **AT-BLC** achieve nearly uniform PIM processing rates across multiple MCs, minimizing synchronization overhead in multi-MC systems. Figure 9 shows the standard deviation of PIM completion times across MCs in the LPDDR-PIM system during concurrent execution of CPU and PIM workloads. With bandwidth-sensitive workloads (CPU-hi), **FRFCFS**, **Batch**, and **TCM** exhibit significant MC-to-MC variation, with standard deviations exceeding 50K cycles, whereas **ST-BLC** and **AT-BLC** maintain tight synchronization. Under latency-sensitive workloads (CPU-low), all schedulers except TCM show stable performance, while TCM always prioritizes CPU requests, delaying PIM execution and increasing MC-to-MC imbalance.

7.3 Memory Cycle Breakdown for AT-BLC and ST-BLC

To further compare **AT-BLC** and **ST-BLC**, we conducted a memory cycle breakdown analysis using **AT-BLC** and **ST-BLC** with varying *N* and *P* values, considering the DRAM row size of 64 columns. (Figure 10(a)). To account for different input scenarios, we varied the number of CPU programs (510.parest) in an LPDDR-PIM system and evaluated performance using GPT-3 6.7B and XL models.

AT-BLC effectively monitors the progress of PIM operations and dynamically adjusts its threshold, allowing it to operate efficiently across diverse scenarios. In all cases, it satisfies the PIM target performance. When the demand for PIM execution is high, **AT-BLC** maintains sufficient PIM cycles even as CPU bandwidth demand increases. Conversely, when fewer PIM operations are required, it allocates more cycles to servicing CPU memory requests. In contrast, the optimal *N* and *P* values for **ST-BLC** vary depending on the size of the PIM workload and the memory intensity of the CPU workload. For example, under GPT-3 6.7B with highly memory-intensive CPU workloads, larger *N* values overly favor CPU memory requests, leading to failure in meeting the PIM QoS target. On the other hand, when the PIM workload is lighter GPT-3 XL, fewer PIM operations are required, and even higher *N* values can meet QoS. Reducing the absolute values of **N** and **P**–even if their ratio remains constant–results in more frequent PIM-ACT commands, which significantly degrade system performance.

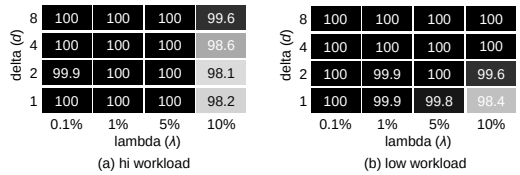


Figure 11: Percentage of PIM operations meeting QoS requirements with AT-BLC, varying *d* and λ, where GPT-3 6.7B is executed with (a) memory-intensive or (b) non-intensive CPU workloads together.

Figure 10(b) shows our further extended cycle breakdown experiment to compare **AT-BLC** and **ST-BLC** with and without AAO. With AAO, the cycle for PIM-ACT slightly decreases compared to the naive version. This allows more cycles to be spent on useful work—namely, PIM and CPU execution—leading to a system performance improvement of approximately 1.2% to 1.5% for both ST-BLC and AT-BLC. In **ST-BLC**, the gain is shared across PIM and CPU workloads, with respective cycle contributions of 0.75% and 0.45% when running 16 CPU programs. In contrast, **AT-BLC** prioritizes CPU execution once PIM QoS is satisfied. As a result, while the PIM cycle ratio remains unchanged, the CPU cycle alone increases by 1.45%. This shows that AT-BLC efficiently allocates bandwidth, ensuring reasonable CPU performance once PIM QoS is met.

7.4 Sensitivity Study on AT-BLC

Figure 11 illustrates the proportion of GEMV operations within the PIM workload that meets QoS requirements under varying d and λ values when using AT-BLC in the LPDDR-PIM system. GPT-3 6.7B, which requires lots of GEMV operations, is used as the PIM workload in this experiment. Larger delta values generally lead to a slightly better ratio of QoS satisfaction by responding more quickly to changes in memory intensity. However, as AT-BLC updates the N value at short intervals (i.e., refresh interval), the impact of d on overall performance is limited. In contrast, λ, which determines whether an update is performed, affects QoS adherence when set too high. A conservative update caused by a large λ value results in failing to meet the target latency. Therefore, we set d and λ to 8 and 0.1%, respectively. Note that these parameters can be tuned according to the operational characteristics of the PIM devices.

8 Related Works

Efficient PIM command processing with macro instruction: To mitigate the burden on the core caused by issuing repeated PIM commands, prior works proposed utilizing macro instructions that encapsulate multiple micro commands [27, 32, 48, 54]. For example, TRiM [48] and Darwin [32] incorporate instruction decoders and generators within memory nodes to offload PIM command issuance. However, this design prevents the MC from tracking the bank state, resulting in normal memory requests being unable to proceed concurrently and remaining pending until PIM execution is completed. Similarly, AESPA [27] introduces a long-data command, which enables PIM operations at the DRAM row level with deterministic execution latency. However, it lacks consideration for the limited memory command space defined by JEDEC standards, making it difficult to be compatible with various systems.

PIM command scheduling with normal memory accesses: Prior works [7, 17, 47, 54] focused on workload or application-level scheduling techniques for PIM that optimize batching and pipelining, while command-level scheduling between PIM and normal memory accesses remains relatively underexplored. One attempt to address this issue [41] proposed gather & issue scheduling policy which triggers PIM execution once the number of accumulated PIM requests exceeds a predefined threshold. However, this approach can lead to inefficient mode switching if the threshold selection does not account for DRAM architectural characteristics, such as the number of DRAM columns, thereby negatively impacting system efficiency. Moreover, even with an appropriately chosen threshold, when the PIM workload intensity is high, regular memory requests experience excessive queuing delays, potentially leading to QoS degradation. A recent work [15] proposes a PIM-aware MC that separates PIM and regular memory requests using virtual channels and schedules them based on arbitration targeting fairness and throughput. This approach is conceptually similar to our ST-BLC, as both coordinate priorities between PIM and normal memory accesses. However, our work goes further by introducing AT-BLC that adjusts scheduling dynamically to meet QoS targets, and by proposing a unified protocol interface (PIM-ACT) for compatibility across various PIM architectures.

9 Conclusion

We have proposed ComPASS, a novel PIM protocol and scheduling solution designed to address the challenges of compatibility and efficiency in PIM-attached systems. ComPASS introduces the PIM-ACT command, which enables multi-bank activation by leveraging the unused memory command space in JEDEC standards, and provides corresponding system support to ensure seamless integration of PIM operations. It also supports architecture-aware optimizations through per-device customization, improving performance without sacrificing protocol compatibility. Through case studies with HBM-PIM and GDDR6-AiM, we demonstrate that existing PIM architectures can be adapted to ComPASS without requiring fundamental design changes. To further enhance PIM system efficiency, we introduce static and adaptive throughput balancer scheduling solutions, which effectively manage scheduling between PIM and conventional memory requests. These solutions ensure consistent PIM performance while serving memory requests from general workloads within a reasonable latency. Our evaluation using GEMV operations for GPT-3 inference workloads demonstrates that LPDDR-PIM with ComPASS reliably meets QoS targets for token generation, improving the GEMV performance by 10.75× compared to a system without PIM, even when executed concurrently with memory-intensive SPEC CPU workloads. These results highlight ComPASS's potential to enhance the compatibility of future PIM-integrated architectures.

Acknowledgments

This work was supported by Institute of Information & communications Technology Planning & Evaluation (IITP) grant funded by the Korea government (MSIT) (No.RS-2025-02263669). Eojin Lee is the corresponding author.

References

[1] Junwhan Ahn, Sungpack Hong, Sungjoo Yoo, Onur Mutlu, and Kiyoung Choi. 2015. A Scalable Processing-in-Memory Accelerator for Parallel Graph Processing. In *Proceedings of the 42nd Annual International Symposium on Computer Architecture*.

[2] Hadi Asghari-Moghaddam, Young Hoon Son, Jung Ho Ahn, and Nam Sung Kim. 2016. Chameleon: Versatile and Practical Near-DRAM Acceleration Architecture for Large Memory Systems. In *Proceedings of the 49th Annual IEEE/ACM International Symposium on Microarchitecture*.

[3] Scott Beamer, Krste Asanovic, and David A. Patterson. 2015. The GAP Benchmark Suite. *arxiv* (2015).

[4] Sid Black, Stella Biderman, Eric Hallahan, Quentin Anthony, Leo Gao, Laurence Golding, Horace He, Connor Leahy, Kyle McDonell, Jason Phang, Michael Pieler, USVSN Sai Prashanth, Shivanshu Purohit, Laria Reynolds, Jonathan Tow, Ben Wang, and Samuel Weinbach. 2022. GPT-NeoX-20B: An Open-Source Autoregressive Language Model. *arXiv* (2022).

[5] Jay B Brockman, Shyamkumar Thoziyoor, Shannon K Kuntz, and Peter M Kogge. 2004. A Low Cost, Multithreaded Processing-in-Memory System. In *Proceedings of the 3rd Workshop on Memory Performance Issues*.

[6] Tom B Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel M. Ziegler, Jeffrey Wu, Clemens Winter, Christopher Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. 2020. Language Models are Few-shot Learners. *arXiv* (2020).

[7] Shuang Chen, Yi Jiang, Christina Delimitrou, and José F Martínez. 2022. PIM-Cloud: QoS-Aware Resource Management of Latency-Critical Applications in Clouds with Processing-in-Memory. In *Proceedings of the 28th IEEE International Symposium on High-Performance Computer Architecture*. IEEE.

[8] Ping Chi, Shuangchen Li, Cong Xu, Tao Zhang, Jishen Zhao, Yongpan Liu, Yu Wang, and Yuan Xie. 2016. PRIME: A Novel Processing-in-Memory Architecture for Neural Network Computation in ReRAM-Based Main Memory. In *Proceedings of the 43rd Annual International Symposium on Computer Architecture*.

[9] The Linux Kernel Development Community. 2024. *HugeTLB subsystem*. https://www.kernel.org/doc/html/latest/admin-guide/mm/hugetlbpage.html

[10] Fabrice Devaux. 2019. The True Processing in Memory Accelerator. In *IEEE Hot Chips 31 Symposium*.

[11] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2018. BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. *arXiv* (2018).

[12] Fei Gao, Georgios Tziantzioulis, and David Wentzlaff. 2019. ComputeDRAM: In-Memory Compute Using Off-The-Shelf Drams. In *Proceedings of the 52nd Annual IEEE/ACM International Symposium on Microarchitecture*.

[13] Mel Gorman and Andy Whitcroft. 2007. Supporting the Allocation of Large Contiguous Regions of Memory. In *Proceedings of the Ottawa Linux Symposium*.

[14] Yufeng Gu, Alireza Khadem, Sumanth Umesh, Ning Liang, Xavier Servot, Onur Mutlu, Ravi Iyer, , and Reetuparna Das. 2025. PIM is All You Need: A CXL-Enabled GPU-Free System for LLM Inference. In *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*.

[15] Sudhanshu Gupta, Niti Madan, Sooraj Puthoor, Nuwan Jayasena, and Sandhya Dwarkadas. 2025. Concurrent PIM and Load/Store Servicing in PIM-Enabled Memory. In *Proceedings of 2025 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*.

[16] Mingxuan He, Choungki Song, Ilkon Kim, Chunseok Jeong, Seho Kim, Il Park, Mithuna Thottethodi, and TN Vijaykumar. 2020. Newton: A DRAM-maker's Accelerator-in-Memory (AiM) Architecture for Machine Learning. In *Proceedings of the 53rd Annual IEEE/ACM International Symposium on Microarchitecture*.

[17] Guseul Heo, Sangyeop Lee, Jaehong Cho, Hyunmin Choi, Sanghyeon Lee, Hyungkyu Ham, Gwangsun Kim, Divya Mahajan, and Jongse Park. 2024. NeuPIMs: NPU-PIM Heterogeneous Acceleration for Batched LLM Inferencing. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*.

[18] Jaehoon Heo, Junsoo Kim, Sukbin Lim, Wontak Han, and Joo-Young Kim. 2022. T-PIM: An Energy-Efficient Processing-in-Memory Accelerator for End-to-End On-Device Training. *IEEE Journal of Solid-State Circuits* (2022).

[19] JEDEC. 2012. DDR4 SDRAM Standard.

[20] JEDEC. 2015. Annex L: Serial Presence Detect (SPD) for DDR4 SDRAM Modules.

[21] JEDEC. 2015. High Bandwidth Memory DRAM (HBM1, HBM2).

[22] JEDEC. 2020. DDR5 SDRAM Standard.

[23] JEDEC. 2020. Low Power Double Data Rate 5 (LPDDR5).

[24] JEDEC. 2023. Graphics Double Data Rate (GDDR6) SGRAM Standard.

[25] Taeyang Jeong and Eui-Young Chung. 2024. PipePIM: Maximizing Computing Unit Utilization in ML-Oriented Digital PIM by Pipelining and Dual Buffering. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* (2024).

[26] Kyoungho Jeun, Hyeonu Kim, and Eojin Lee. 2016. Fold-PIM: A Cost-Efficient LPDDR5-Based PIM for On-Device SLMs. *IEEE Computer Architecture Letters* (2016).

[27] Hongju Kal, Chanyoung Yoo, and Won Woo Ro. 2023. AESPA: Asynchronous Execution Scheme to Exploit Bank-Level Parallelism of Processing-in-Memory. In *Proceedings of the 56th Annual IEEE/ACM International Symposium on Microarchitecture*.

[28] Byeongho Kim, Sanghoon Cha, Sangsoo Park, Jieun Lee, Sukhan Lee, Shin-haeng Kang, Jinin So, Kyungsoo Kim, Jin Jung, Jong-Geon Lee, Sunjung Lee, Yoonah Paik, Hyeonsu Kim, Jin-Seong Kim, Won-Jo Lee, Yuhwan Ro, YeonGon Cho, Jin Hyun Kim, JoonHo Song, Jaehoon Yu, Seungwon Lee, Jeonghyeon Cho, and Kyomin Sohn. 2024. The Breakthrough Memory Solutions for Improved Performance on LLM Inference. *IEEE Micro* (2024).

[29] Byeongho Kim, Jongwook Chung, Eojin Lee, Wonkyung Jung, Sunjung Lee, Jaewan Choi, Jaehyun Park, Minbok Wi, Sukhan Lee, and Jung Ho Ahn. 2020. MViD: Sparse Matrix-Vector Multiplication in Mobile DRAM for Accelerating Recurrent Neural Networks. *IEEE Trans. Comput.* (2020).

[30] Chang Hyun Kim, Won Jun Lee, Yoonah Paik, Seok Young Kim, and Seon Wook Kim. 2023. BL-PIM: Varying the Burst Length to Realize the All-Bank Performance and Minimize the Multi-Workload Interference for in-DRAM PIM. *IEEE Access* (2023).

[31] Chang Hyun Kim, Won Jun Lee, Yoonah Paik, Kiyong Kwon, Seok Young Kim, Il Park, and Seon Wook Kim. 2021. Silent-PIM: Realizing the Processing-in-Memory Computing With Standard Memory Requests. *IEEE Transactions on Parallel and Distributed Systems* (2021).

[32] Donghyuk Kim, Jae-Young Kim, Wontak Han, Jongsoon Won, Haerang Choi, Yongkee Kwon, and Joo-Young Kim. 2024. Darwin: A DRAM-Based Multi-Level Processing-in-Memory Architecture for Column-Oriented Database. *IEEE Transactions on Emerging Topics in Computing* (2024).

[33] Guhyun Kim, Jinkwon Kim, Nahsung Kim, Woojae Shin, Jongsoon Won, Hyunha Joo, Haerang Choi, Byeongju An, Gyeongcheol Shin, Dayeon Yun, Jeongbin Kim, Changhyun Kim, Ilkon Kim, Jaehan Park, Yosub Song, Byeongsu Yang, Hyeongdeok Lee, Seungyeong Park, Wonjun Lee, Seonghun Kim, Yonghoon Park, Yousub Jung, Gi-Ho Park, and Euicheol Lim. 2024. SK Hynix AI-Specific Computing Memory Solution: From AiM Device to Heterogeneous AiMX-xPU System for Comprehensive LLM Inference. In *2024 IEEE Hot Chips 36 Symposium (HCS)*.

[34] Heesu Kim, Hanmin Park, Taehyun Kim, Kwanheum Cho, Eojin Lee, Soojung Ryu, Hyuk-Jae Lee, Kiyoung Choi, and Jinho Lee. 2021. GradPIM: A Practical Processing-in-DRAM Architecture for Gradient Descent. In *Proceedings of the 27th IEEE International Symposium on High-Performance Computer Architecture*. IEEE.

[35] Jin Hyun Kim, Shin-haeng Kang, Sukhan Lee, Hyeonsu Kim, Woongjae Song, Yuhwan Ro, Seungwon Lee, David Wang, Hyunsung Shin, Bengseng Phuah, Jihyun Choi, Jinin So, YeonGon Cho, JoonHo Song, Jangseok Choi, Jeonghyeon Cho, Kyomin Sohn, Youngsoo Sohn, Kwangil Park, and Nam Sung Kim. 2021. Aquabolt-XL: Samsung HBM2-PIM with In-memory Processing for ML Accelerators and Beyond. In *IEEE Hot Chips 33 Symposium*.

[36] Yoongu Kim, Michael Papamichael, Onur Mutlu, and Mor Harchol-Balter. 2010. Thread Cluster Memory Scheduling: Exploiting Differences in Memory Access Behavior. In *2010 43rd Annual IEEE/ACM International Symposium on Microarchitecture*.

[37] Yoongu Kim, Weikun Yang, and Onur Mutlu. 2016. Ramulator: A Fast and Extensible DRAM Simulator. *IEEE Computer Architecture Letters* (2016).

[38] Yongkee Kwon, Kornijcuk Vladimir, Nahsung Kim, Woojae Shin, Jongsoon Won, Minkyu Lee, Hyunha Joo, Haerang Choi, Guhyun Kim, Byeongju An, Jeongbin Kim, Jaewook Lee, Ilkon Kim, Jaehan Park, Chanwook Park, Yosub Song, Byeongsu Yang, Hyungdeok Lee, Seho Kim, Daehan Kwon, Seongju Lee, Kyuyoung Kim, Sanghoon Oh, Joonhong Park, Gimoon Hong, Dongyoon Ka, Kyudong Hwang, Jeongje Park, Kyeongpil Kang, Jungyeon Kim, Junyeol Jeon, Myeongjun Lee, Minyoung Shin, Minhwan Shin, Jaekyung Cha, Changson Jung, Kijoon Chang, Chunseok Jeong, Euicheol Lim, Il Park, Junhyun Chun, and Sk Hynix. 2022. System Architecture and Software Stack for GDDR6-AiM. In *IEEE Hot Chips 34 Symposium*.

[39] Jimin Lee and Soonhoi Ha. 2025. Empowering Edge Devices With Processing-In-Memory for On-Device Language Inference. *IEEE Embedded Systems Letters* (2025).

[40] Sukhan Lee, Shin-haeng Kang, Jaehoon Lee, Hyeonsu Kim, Eojin Lee, Seungwoo Seo, Hosang Yoon, Seungwon Lee, Kyounghwan Lim, Hyunsung Shin, Jinhyun Kim, O Seongil, Anand Iyer, David Wang, Kyomin Sohn, and Nam Sung Kim. 2021. Hardware Architecture and Software Stack for PIM Based on Commercial DRAM Technology : Industrial Product. In *Proceedings of the 48th Annual International Symposium on Computer Architecture*.

[41] Seungyong Lee, Sanghyun Lee, Minseok Seo, Chunmyung Park, Hyuk-Jae Lee, Woojae Shin, and Hyun Kim. 2021. A High-Performance Scheduling Algorithm for Mode Transition in PIM. In *IEEE International Conference on Consumer Electronics-Asia*. IEEE.

[42] Hanchen Li, Yuhan Liu, Yihua Cheng, Siddhant Ray, Kuntai Du, and Junchen Jiang. 2024. Chatterbox: Robust Transport for LLM Token Streaming under Unstable Network. *arXiv* (2024).
[43] Haocong Luo, Yahya Can Tuğrul, F Nisa Bostancı, Ataberk Olgun, A Giray Yağlıkçı, and Onur Mutlu. 2023. Ramulator 2.0: A modern, modular, and extensible dram simulator. *IEEE Computer Architecture Letters* (2023).
[44] Onur Mutlu and Thomas Moscibroda. 2008. Parallelism-aware Batch Scheduling: Enhancing both Performance and Fairness of Shared DRAM Systems. In *Proceedings of the 35th Annual International Symposium on Computer Architecture*.
[45] Geraldo F Oliveira, Ataberk Olgun, Abdullah Giray Yağlıkçı, F Nisa Bostancı, Juan Gómez-Luna, Saugata Ghose, and Onur Mutlu. 2024. MIMDRAM: An End-to-End Processing-Using-DRAM System for High-Throughput, Energy-Efficient and Programmer-Transparent Multiple-Instruction Multiple-Data Computing. In *Proceedings of the 30th IEEE International Symposium on High-Performance Computer Architecture*.
[46] Cristobal Ortega, Yann Falevoz, and Renaud Ayrignac. 2024. PIM-AI: A Novel Architecture for High-Efficiency LLM Inference. *arXiv* (2024).
[47] Jaehyun Park, Jaewan Choi, Kwanhee Kyung, Michael Jaemin Kim, Yongsuk Kwon, Nam Sung Kim, and Jung Ho Ahn. 2024. AttAcc! Unleashing the Power of PIM for Batched Transformer-based Generative Model Inference. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*.
[48] Jaehyun Park, Byeongho Kim, Sungmin Yun, Eojin Lee, Minsoo Rhu, and Jung Ho Ahn. 2021. TRiM: Enhancing processor-memory interfaces with scalable tensor reduction in memory. In *Proceedings of the 54th Annual IEEE/ACM International Symposium on Microarchitecture*.
[49] Sang-Soo Park, KyungSoo Kim, Jinin So, Jin Jung, Jonggeon Lee, Kyoungwan Woo, Nayeon Kim, Younghyun Lee, Hyungyo Kim, Yongsuk Kwon, Jinhyun Kim, Jieun Lee, YeonGon Cho, Yongmin Tai, Jeonghyeon Cho, Hoyoung Song, Jung Ho Ahn, and Nam Sung Kim. 2024. An LPDDR-based CXL-PNM Platform for TCO-efficient Inference of Transformer-based Large Language Models. In *Proceedings of the 30th IEEE International Symposium on High-Performance Computer Architecture*.
[50] Ben Perach, Ronny Ronen, Benny Kimelfeld, and Shahar Kvatinsky. 2023. Understanding Bulk-Bitwise Processing In-Memory Through Database Analytics. *IEEE Transactions on Emerging Topics in Computing* (2023).
[51] Scott Rixner, William J Dally, Ujval J Kapasi, Peter Mattson, and John D Owens. 2000. Memory Access Scheduling. In *Proceedings of the 27th Annual International Symposium on Computer Architecture*.
[52] Sourjya Roy, Mustafa Ali, and Anand Raghunathan. 2021. PIM-DRAM: Accelerating Machine Learning Workloads using Processing in Commodity DRAM. *IEEE Journal on Emerging and Selected Topics in Circuits and Systems* (2021).
[53] Samsung. 2022. PIMSimulator. https://github.com/SAITPublic/PIMSimulator/.
[54] Minseok Seo, Xuan Truong Nguyen, Seok Joong Hwang, Yongkee Kwon, Guhyun Kim, Chanwook Park, Ilkon Kim, Jaehan Park, Jeongbin Kim, Woojae Shin, Jongsoon Won, Haerang Choi, Kyuyoung Kim, Daehan Kwon, Chunseok leong, Sangheon Lee, Yongseok Choi, Wooseok Byun, Seungcheol Baek, Hyuk-Jae Lee, and John Kim. 2024. IANUS: Integrated Accelerator based on NPU-PIM Unified Memory System. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*.
[55] Hyunsung Shin, Dongyoung Kim, Eunhyeok Park, Sungho Park, Yongsik Park, and Sungjoo Yoo. 2018. McDRAM: Low latency and Energy-Efficient Matrix Computations in DRAM. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* (2018).
[56] SPEC. 2017. SPEC CPU2017. https://www.spec.org/cpu2017/.
[57] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, Aurelien Rodriguez, Armand Joulin, Edouard Grave, and Guillaume Lample. 2023. LLaMA: Open and Efficient Foundation Language Models. *arXiv* (2023).
[58] Xinfeng Xie, Zheng Liang, Peng Gu, Abanti Basak, Lei Deng, Ling Liang, Xing Hu, and Yuan Xie. 2021. SpaceA: Sparse Matrix Vector Multiplication on Processing-in-Memory Accelerator. In *Proceedings of the 27th IEEE International Symposium on High-Performance Computer Architecture*.
[59] Jiajun Xu, Zhiyuan Li, Wei Chen, Qun Wang, Xin Gao, Qi Cai, and Ziyuan Ling. 2024. On-Device Language Models: A Comprehensive Review. *arxiv* (2024).
[60] Sungmin Yun, Kwanhee Kyung, Juhwan Cho, Jaewan Choi, Jongmin Kim, Byeongho Kim, Sukhan Lee, Kyomin Sohn, and Jung Ho Ahn. 2024. Duplex: A Device for Large Language Models with Mixture of Experts, Grouped Query Attention, and Continuous Batching. In *Proceedings of the 57th Annual IEEE/ACM International Symposium on Microarchitecture*.
[61] Sungmin Yun, Hwayong Nam, Jaehyun Park, Byeongho Kim, Jung Ho Ahn, and Eojin Lee. 2023. GraNDe: Efficient Near-Data Processing Architecture for Graph Neural Networks. *IEEE Trans. Comput.* (2023).
[62] Aohan Zeng, Xiao Liu, Zhengxiao Du, Zihan Wang, Hanyu Lai, Ming Ding, Zhuoyi Yang, Yifan Xu, Wendi Zheng, Xiao Xia, Weng Lam Tam, Zixuan Ma, Yufei Xue, Jidong Zhai, Wenguang Chen, Peng Zhang, Yuxiao Dong, and Jie Tang. 2022. GLM-130B: An Open Bilingual Rre-trained Model. *arXiv* (2022).
[63] Yilong Zhao, Mingyu Gao, Fangxin Liu, Yiwei Hu, Zongwu Wang, Han Lin, Ji Li, He Xian, Hanlin Dong, Tao Yang, Naifeng Jing, Xiaoyao Liang, and Li Jiang. 2024. UM-PIM: DRAM-based PIM with Uniform & Shared Memory Space. In *Proceedings of the 51st Annual International Symposium on Computer Architecture*.
[64] Minxuan Zhou, Weihong Xu, Jaeyoung Kang, and Tajana Rosing. 2022. TransPIM: A Memory-based Acceleration via Software-Hardware Co-design for Transformer. In *Proceedings of the 28th IEEE International Symposium on High-Performance Computer Architecture*.